PLPDI (and "Extended")
Autumn Semester 2024-25

The University of Birmingham
School of Computer Science
© Sean Moss 2024-25

Exercise Sheet 1 - Functional Programming

Handed out: Monday 30 September
To be completed by: Monday 7 October (not for credit)

This exercise is on functional programming in Haskell. You are encouraged to find a Haskell set up that works for you and to actually try out your programs. Questions that involve lists may be answered with either Haskell's built-in list type or the `IntList` type from lectures (remember to include the `data` declaration in your code). You are not expected to be able to answer all of this sheet without consulting external resources — feel free to use any resources you like to help you understand Haskell features, but try to write the programs yourself.

*Questions marked with * are optional, more difficult challenges.*

Question 1: Datatypes

Consider the following datatype declaration.

```
data Shape = Circle Double | Rectangle Double Double
```

Interpret a `Circle` as storing its radius and a `Rectangle` as storing its height and width. Use pattern-matching to write a function

```
area :: Shape -> Real
```

which computes the area of a `Shape`. You should be able to use your function like this:

```
ghci> area (Circle 1)
3.141592653589793
ghci> area (Rectangle 2 3)
6.0
```

[Hint: you can use `pi` in Haskell without importing anything.]

Question 2: Inductive datatypes

Consider the following datatype declaration.

```
data MixList = MNil | IntCons Int MixList | BoolCons Bool MixList deriving Show
```

(The `deriving Show` is just to allow you to print `MixList` values). Explain why the values of this type look like lists that can contain both integer and boolean values. Consider the following function

```
squareDrop :: MixList -> MixList
squareDrop MNil = MNil
squareDrop (IntCons n ms) = IntCons (n * n) (squareDrop ms)
squareDrop (BoolCons _ ms) = squareDrop ms
```

which squares all of the integer entries and drops all of the boolean entries. Based on this example, write a function `dropNot` which drops all of the integer values and performs boolean 'not' on the boolean entries.

*Rewrite `squareDrop` to a function `mapDrop` that takes an extra argument of type `Int -> Int` and applies that to each integer entry instead of squaring. Thus you will recover `squareDrop` = `mapDrop` (`\n -> n * n`).

Question 3: Negate a list

```
negateAll :: [Integer] -> [Integer]
```

Given a list of integers l, define a function that returns a new list with all the elements of l with sign negated, i.e., positive integers are to become negative and negative integers positive. Example:

```
[2, -5, 8, 0] ==> [-2, 5, -8, 0]
```

Question 4: Checking for an element

Reimplement the built-in function `elem`. That is, define a function

```
myElem :: Integer -> [Integer] -> Bool
```

which, given an integer x and a list l, checks whether x occurs in l. Examples:

```
myElem 3 [7, 5, 3, 8]   ==>   True
myElem 7 [7, 5, 3, 8]   ==>   True
myElem 6 [7, 5, 3, 8]   ==>   False
```

Question 5: Safely searching for an element*

As a variant of the previous problem, define:

```
find :: Integer -> [Integer] -> Maybe Integer
```

which returns `Nothing` if the the integer does not occur in the list and returns `Just n` if the integer first occurs in the list at position `n`. Examples:

```
find 3 [7, 5, 3, 8]   ==>   Just 2
find 7 [7, 5, 3, 8]   ==>   Just 0
find 6 [7, 5, 3, 8]   ==>   Nothing
```

Question 6: Check for positives

```
allPositive : [Integer] -> Bool
```

Given a list of integers l, return a boolean value indicating whether *all* its elements are positive, i.e., > 0.

Question 7: Find the positives

```
positives :: [Integer] -> [Integer]
```

Given a list of integers l, return a new list which contains all the positive elements of l. The elements should appear in the result in the same relative order as in l. Example:

```
[2, 3, -5, 8, -2]   ==>   [2, 3, 8]
```

Question 8: Inline pattern-matching*

Consider the function `g` defined by the following Haskell expression.

```
g m = let f xs ys = case xs of { [] -> ys; (x:xs) -> f xs (x:ys) } in f m []
```

Work out what this function does (try running it if you get stuck). Rewrite it to use top-level definitions and pattern-matching. (No `let`'s or `case`'s).

Question 9: Sortedness

```
sorted :: [Integer] -> Bool
```

Given a list of integers l, this function should return a boolean value indicating whether l is sorted in ascending order. (There can be duplicate copies of elements. But, sortedness would require that all the duplicate copies would appear together.)

Question 10: Enumerations*

Write a Haskell expression producing the same output as `[0..]`.